COMPLETE STUDY GUIDE · VIVA · PROJECT REVIEW · INTERVIEW

Predictive Memory Paging System with Workload Awareness

How the project works, why it is built the way it is, and how to explain and defend it

G.K Shreyas Bharadwaj (24BCE2256) · Vaastav Bajanthri (24BCE2223) · P. Giphy Bershan (24BCE2201)

Version covered: the repository as of 19 October 2026. Final system = `paging.final.run_final` (Phases 14-17). Every code snippet is read from the source files, every worked example is computed by running the real code, and every result is read from `results/*.csv`. Regenerate with `python scripts/19_study_guide_pdf.py`.

The project in one paragraph

A trace-driven simulator of operating-system paging. A **predictor** guesses which memory pages a program will need next; a **decision engine** decides whether loading a guess early (prefetching) is worth the risk, using feedback on whether page faults are actually going down. On 22 real benchmark programs the final system removes **55.0%** of LRU's page faults on average (Linux-style readahead: 50.7%). In a time model it is about 1.43x faster than LRU on a parallel SSD (readahead 1.44x) and slower than LRU on a hard disk.

Contents

1. Project overview 4

1.1 What the project is 4

1.2 The problem it solves 4

1.3 Objectives and expected output 4

1.4 A real-world analogy 5

1.5 What is useful or different 5

2. Core concepts and fundamentals 6

3. Complete project architecture 10

3.1 Components, inputs and outputs 10

3.2 How the components communicate 11

3.3 Technologies in the architecture 11

4. Complete working and execution flow 12

4.1 From start to output 12

4.2 The per-request loop 12

4.3 What happens in unusual situations 13

4.4 What the user sees 14

5. Complete logic explanation 15

5.1 Installing a page (the single entry point) 15

5.2 Handling a request 15

5.3 The decision rule 16

5.4 The feedback controller 16

5.5 Three ways to estimate the fault delta 17

5.6 Predictor logic 18

5.7 Eviction policies in one table 20

6. Source code explanation 21

6.1 Folder structure 21

6.2 Responsibilities of the important files 21

6.3 How the scripts use the library 27

7. Algorithms and mathematical logic 28

7.1 Formulas 28

7.2 Complexity and memory 28

7.3 Assumptions and limits of the maths 29

7.4 Implemented vs only discussed 29

8. Technologies, tools and dependencies 30

9. Data flow and worked examples 31

Example A: three eviction policies, 3 frames 31

Example B: Markov order-1 learning a pattern . . . 31
Example C: readahead window growth . . . 32
Example D: delta-Markov predicts pages never seen . . . 32
Example E: a full prefetching run, step by step . . . 33
Example F: the controller over five epochs . . . 33
Example G: the time model . . . 34
10. Testing, evaluation and results . . . 35
10.1 How the project is tested . . . 35
10.2 Results . . . 35
10.3 Every component side by side . . . 37
10.4 What the results show and do not show . . . 37
10.5 Known issues and unverified claims . . . 38
11. Novelty and design decisions . . . 39
11.1 What is genuinely ours and what is borrowed . . . 39
11.2 Design decisions, alternatives and evidence . . . 39
11.3 Trade-offs . . . 39
12. Limitations and future improvements . . . 40
12.1 Current limitations (existing system) . . . 40
12.2 Suggested improvements (not implemented) . . . 40
13. Important questions and answers . . . 41
14. Quick revision . . . 43
14.1 One-page summary . . . 43
14.2 Working flow in numbered steps . . . 43
14.3 Concepts to remember . . . 43
14.4 Key formulas and metrics . . . 43
14.5 Component → responsibility . . . 44
14.6 Most important files and functions . . . 44
14.7 Novelty in two sentences . . . 44
14.8 Two-minute explanation . . . 45
Glossary . . . 46
References and sources . . . 46

CHAPTER 1

1. Project overview

1.1 What the project is

A computer program keeps the data it is working on in **RAM** (fast, small) and the rest on a **disk** (slow, large). Data moves between them in fixed blocks called **pages** (4 KB). When a program asks for a page that is not in RAM, it stops and waits while the operating system reads it from disk: a **page fault**. This project is a **simulator** written in Python that replays recorded page requests of real programs and tests ideas for reducing page faults by **predicting** the next pages and loading them early (**prefetching**).

1.2 The problem it solves

Classical paging is reactive: it waits for a fault, then loads the page and, if RAM is full, removes another page chosen by an **eviction policy** (FIFO, LRU, Clock). Even the theoretically best eviction policy (Belady's OPT) still pays for every first visit to a page. Prefetching can avoid those, but it is risky: when RAM is full, every prefetched page pushes another page out, and a wrong guess can *create* a page fault. The project asks:

- How should a system **decide** when a prediction is worth acting on, and which feedback signal tells it whether prefetching is helping?
- Which predictors work on real programs, and how do they compare with what Linux already does (readahead)?
- Does a lower page-fault count actually mean the program finishes sooner?

1.3 Objectives and expected output

Objective	Delivered as
Simulate RAM, eviction policies and prefetching correctly and reproducibly	engine.py, simulate.py, 24 automatic tests
Build lightweight predictors (Markov, delta-Markov, stride, readahead)	predictors.py
Build a decision engine that limits harm using measured feedback	deciders.py (9 engines)
Compare fairly with FIFO, LRU, Clock, ARC, LRU-2, OPT and Linux readahead	scripts 04-11, results/*.csv
Estimate real time, not only fault counts	timing.py, script 14
Explain everything honestly	results/report.html, docs/paper_draft.md, this guide

The **output** of a run is a set of numbers per configuration: page faults, prefetches issued and used, and estimated run time, written to CSV files and collected in an HTML report.

1.4 A real-world analogy

> A student works at a small **desk** (RAM) in a big **library** (disk). Books on the desk can be read instantly; fetching one from the shelves takes a long walk (a page fault). When the desk is full, a book must go back (eviction). A **helper** (predictor) brings books the student will probably need next (prefetching). A good helper saves trips; a bad helper fills the desk with useless books and pushes useful ones back to the shelves. So a **supervisor** (decision engine) checks every week whether the student walked less than they would have without the helper (the fault delta) and tells the helper to be bolder or more careful.

1.5 What is useful or different

- It evaluates **which feedback signal** a cost-aware prefetcher should use, and shows that two plausible signals (confidence, precision) mislead.
- It replaces an impossible "oracle" with a cheap, exact **shadow directory** (page names only), which can be sampled to an eighth of its size.
- It combines a pattern-learning predictor with Linux-style readahead and shows where each one helps.
- It reports that **fewer faults is not the same as faster** (timeliness, extra reads, disk type), which many prefetching studies do not measure.
- It does **not** invent new building blocks: Markov prefetching (1997), readahead, Page Fault Frequency and ghost lists are known. The contribution is the combination and the careful evaluation (Chapter 11).

CHAPTER 2

2. Core concepts and fundamentals

Each concept below is defined before it is used in later chapters. Read them in order: each builds on the previous ones.

Page, RAM, disk

What it is	A **page** is a 4 KB block of memory. **RAM** holds a limited number of pages (**frames**); the **disk** holds all of them.
Why we need it	Everything in the project is measured in pages and frames.
How it works	A memory address is turned into a page number by dropping its last 12 bits (2^{12} = 4,096 bytes).
Example	Address 0x7f3a2c18 → page 0x7f3a2. Addresses 0x7f3a2000 to 0x7f3a2fff all belong to that page.
Connects to	Traces (Stage 1) are lists of page numbers; the memory engine stores a set of resident pages.

Page fault

What it is	A request for a page that is not resident in RAM. The program waits for the disk.
Why we need it	It is the main cost the project tries to reduce.
How it works	The memory engine checks `page in resident`; if not, it counts a fault and installs the page.
Example	RAM holds {1, 2, 3}; a request for page 4 is a fault; a request for 2 is a hit.
Connects to	Every metric (vs_LRU, speed-up) is built on fault counts.

Compulsory (first-touch) fault

What it is	The fault caused by the very first request for a page. No eviction policy can avoid it.
Why we need it	It explains why prefetching can beat OPT.
How it works	A page never seen before cannot already be in RAM unless something loaded it early.
Example	A program that reads 3,000 new pages once has 3,000 compulsory faults under any eviction policy.
Connects to	OPT, readahead, delta-Markov.

Working set

What it is	The set of distinct pages a program uses.
Why we need it	RAM is sized as a fraction of it (1%, 5%), because with ample RAM nothing is evicted and every policy looks identical.
How it works	Count distinct page numbers in the trace.
Example	leslie3d uses 2,134 distinct pages; 1% of that is 21 frames.
Connects to	Experiment configuration in every script.

Eviction policy

What it is	The rule that chooses which page leaves when RAM is full.
Why we need it	Prefetching runs on top of it: the predictor decides what arrives, the policy decides what leaves.
How it works	Each policy keeps its own bookkeeping (order of arrival, last use, frequency) and returns a victim when asked.
Example	FIFO removes the oldest arrival; LRU the least recently used; OPT the page needed furthest in the future (Chapter 7, example A).
Connects to	engine.py policies; MemoryEngine calls choose_victim.

Belady's OPT

What it is	The optimal eviction policy: evict the page whose next use is furthest away.
Why we need it	It is the yardstick: no eviction policy can do better.
How it works	Impossible in a real system (it needs the future) but possible in a simulator, because the trace is recorded.
Example	On [1,2,3,1,4,1,2,5,1,2] with 3 frames: OPT 5 faults, LRU 6, FIFO 7 (computed).
Connects to	OPTFast in engine.py; "headroom" metric.

Prefetching

What it is	Loading a page before it is requested.
Why we need it	Avoids faults, including compulsory ones, if the guess is right and arrives in time.
How it works	After each request the predictor proposes candidates; approved ones are installed through the same path as demand faults.
Example	Loop of 5 pages, 4 frames: LRU faults 15 times, with prefetching 5 times (Chapter 9, example D).
Connects to	Predictor → decision engine → MemoryEngine.prefetch.

Trace-driven simulation

What it is	Replaying a recorded list of page requests through a model of the system instead of running a real kernel.
Why we need it	Identical input for every method (fair), reproducible, and OPT needs the recorded future.
How it works	Read the trace; feed requests one by one into the engine; count events.
Example	Every policy sees exactly the same 200,000 requests of leslie3d.
Connects to	traces.py, simulate.py.

Markov-chain predictor

What it is	A table of "after this context, the next page was X" counts.
Why we need it	Learns repeating patterns from the request stream with almost no cost per request.
How it works	Context = last 1 (order 1) or 2 (order 2) pages. Update: increase the count of the observed next page. Predict: the most frequent next page; confidence = its share of the counts.
Example	After learning 10→20→30 twice, seeing 10 predicts 20 with confidence 1.0 (Chapter 9, example B).
Connects to	MarkovPredictor; part of HybridPredictor and DeltaHybridPredictor.

Stride and delta prediction

What it is	**Stride**: if the last gaps between pages are equal, continue the gap. **Delta-Markov**: a Markov table over gaps instead of pages.
Why we need it	A page-number table cannot predict pages it has never seen; gaps repeat even when pages are new.
How it works	Stride: gaps 8, 8, 8 → next = last + 8. Delta: context = last two gaps, successor = next gap.
Example	Pages 1000, 1001, 1004, 1005, 1008 (gaps +1, +3, +1, +3): delta-Markov predicts 1009, 1012, ... (Chapter 9).
Connects to	StridePredictor, DeltaMarkovPredictor.

Readahead

What it is	The Linux page-cache method: when pages are read in order, fetch a window of the next few; double the window while the run continues.
Why we need it	Covers sequential reading of new data, which Markov cannot.
How it works	Two consecutive pages start a stream; when the program reaches the middle of the fetched window, fetch the next window, twice as large (cap: a quarter of RAM).
Example	100, 101 → fetch 102-103; at 102 → fetch 104-107; at 105 → fetch 108-115 (computed, Chapter 9).
Connects to	ReadaheadPredictor; combined in StreamMarkovPredictor.

Confidence and threshold

What it is	Confidence: how sure the predictor is (0-1). Threshold: the minimum confidence the decision engine accepts.
Why we need it	A way to act only on strong guesses.
How it works	decide(): if RAM is full, accept only if confidence ≥ threshold.
Example	With 2 kept successors, confidence is never below 0.5, so a 0.3 threshold accepts everything (a structural fact found in Phase 7).
Connects to	deciders.py; readahead confidence run/(run+1).

Fault delta (feedback)

What it is	Faults without prefetching minus faults with prefetching, per period (epoch).
Why we need it	The direct measure of whether prefetching helps. Positive = helping.
How it works	Measured every epoch; the controller lowers the threshold when it is positive, raises it when negative (Chapter 5).
Example	Shadow 130 faults, real 100, epoch 1,000 → delta rate +0.03 → bolder.
Connects to	CostAwareFaultDelta._adapt and its three estimators.

Oracle, ghost list, shadow directory

What it is	Three ways to obtain the "without prefetching" count: a full second copy of memory (oracle); a list of pages a prefetch evicted (ghost); a page-ID-only copy of the policy (directory).
Why we need it	A real kernel cannot keep a full second copy; the cheaper estimates make the idea deployable.
How it works	Oracle and directory run a no-prefetch engine on the same requests; the ghost list counts quick refaults of pages a prefetch evicted.
Example	Ghost on clustered@9: said "winning" (+988) when the truth was −1,250; directory: exact.
Connects to	run_prefetch_sim, run_prefetch_sim_ghost, run_prefetch_sim_dir.

Hash sampling (SHARDS)

What it is	Track only pages whose hash falls below a fraction s, with a directory of s × frames entries, then scale counts by 1/s.
Why we need it	Shrinks the shadow directory (idea from Waldspurger et al., FAST 2015).
How it works	sampled(page) = (page × 2654435761 mod 2^{32}) < s × 2^{32}.
Example	With s = 1/8, of pages 1-40 the sampled ones are [5, 13, 18, 26, 34, 39] (computed).
Connects to	simulate.sampled, CostAwareDirectory(sample=...).

Warm-up split

What it is	The first 25% of each trace, in time order.
Why we need it	The only data allowed for choosing settings; prevents tuning on the test data.
How it works	Trace.split = int(len × 0.25); sweeps build Trace(pages[:split]).
Example	leslie3d: requests 0-49,999 are warm-up.
Connects to	Scripts 15, 16 and every design choice in Phases 9-17.

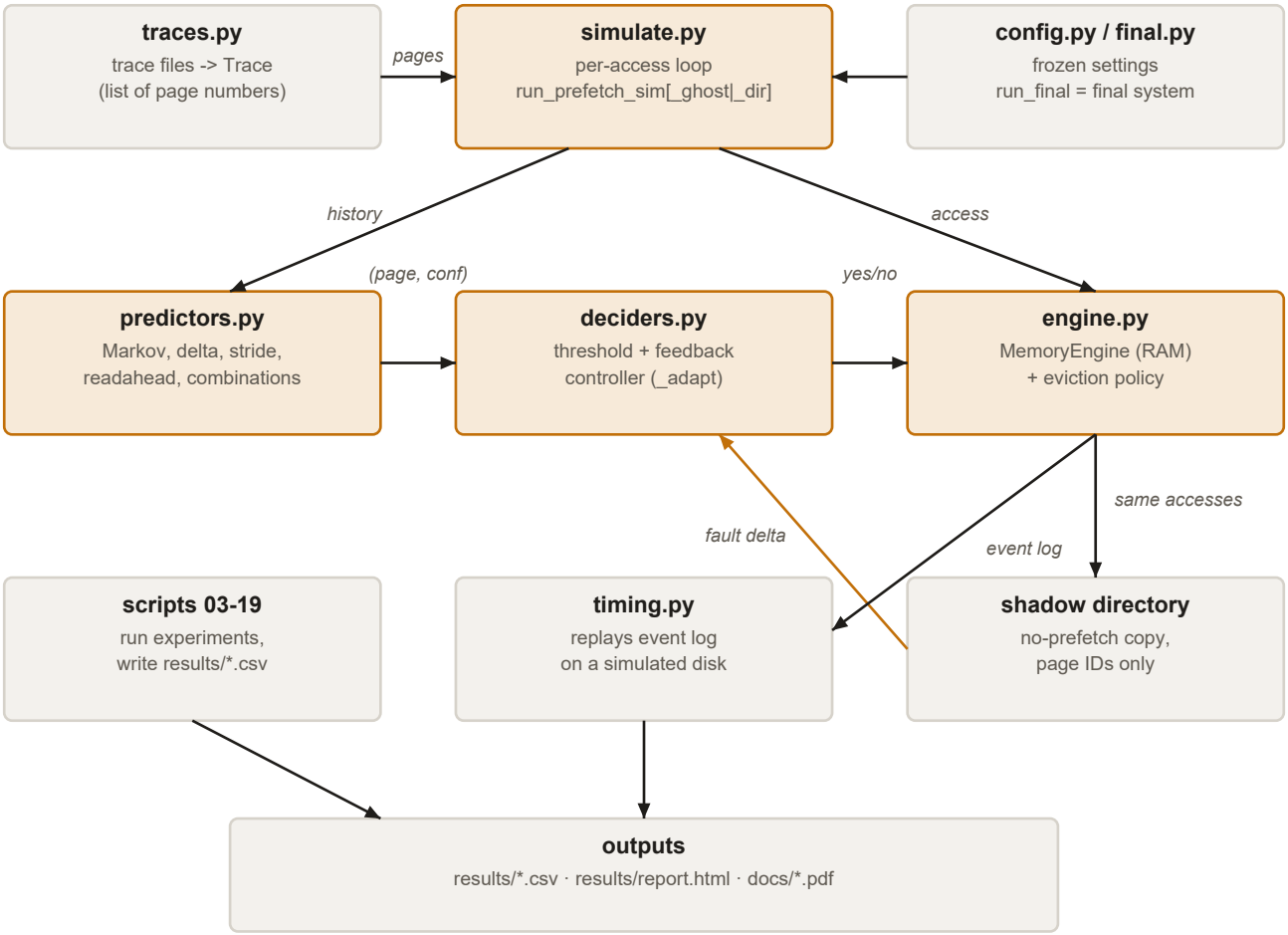
Time model

What it is	A replay of a run's events on a simulated disk to estimate run time.
Why we need it	Fault counts ignore late prefetches, extra reads and queueing.
How it works	Compute time per request, read time per page, parallel channels, blocking faults, non-blocking prefetches that stall if late.
Example	A prefetch issued at time 0.1 with read time 1.0, used at time 0.2 → 0.9 of waiting (computed).
Connects to	timing.replay, script 14.

CHAPTER 3

3. Complete project architecture

The project is a Python library (`src/paging`) plus one script per experiment (`scripts/`). The diagram shows the modules and what flows between them; highlighted boxes are the core per-request loop.



3.1 Components, inputs and outputs

Component (file)	Purpose	Receives	Produces
traces.py	Create, convert and load traces	generator parameters or raw .txt.xz files	Trace objects (page arrays, gaps, 25% split)
engine.py · MemoryEngine	Simulate RAM; the only way pages enter RAM (_install)	access(page, i), prefetch(page, i)	fault/hit, counters, optional event log
engine.py · policies	Choose the victim (FIFO, LRU, Clock, OPT, ARC, LRU-2)	on_insert / on_hit / on_evict notifications	choose_victim → page to evict
predictors.py	Guess the next pages	history (last 8 pages), the actual next page	list of (page, confidence)
deciders.py	Accept or reject each guess; adapt the threshold	(page, confidence, RAM state); fault-delta estimates	yes/no; threshold history
simulate.py	Run the per-request loop with the right feedback	trace, frames, policy class, predictor, decider	statistics dict (faults, prefetches, epoch log...)
timing.py	Estimate time	event log, read time, channels, compute time	total time and its parts
final.py · config.py	Frozen settings and the final system in one call	trace, frames	statistics of the final system
scripts 01-19	Experiments, report and documents	traces, library	results/*.csv, report.html, PDFs

3.2 How the components communicate

1. The simulator owns one **MemoryEngine** with one policy object, one **predictor** and one **decider**. They never call each other directly: the simulator passes data between them.
2. The predictor only ever sees the request history; it never sees RAM. The decider only sees `(page, confidence, state)`; it never sees the predictor's tables. This separation is what lets 9 deciders be compared on byte-identical predictions.
3. Approved prefetches go through `MemoryEngine.prefetch → _install`, the same path as a demand fault, so a broken predictor can slow the system but never corrupt it.
4. For feedback, the simulator runs a second engine on the same requests (oracle or shadow directory) or reads the ghost-list counters, and hands an estimate to the decider once per epoch.
5. If an event list is passed as `log=`, the engine records every fault (F), prefetch (P) and first use of a prefetched page (U); timing.py replays it.

3.3 Technologies in the architecture

Python 3.9+ for everything; NumPy and pandas for arrays and tables; matplotlib for figures; pytest for tests; reportlab for PDFs; the standard library (lzma, urllib, csv, collections, heapq, bisect) for decompression, download and data structures. Chapter 8 explains each.

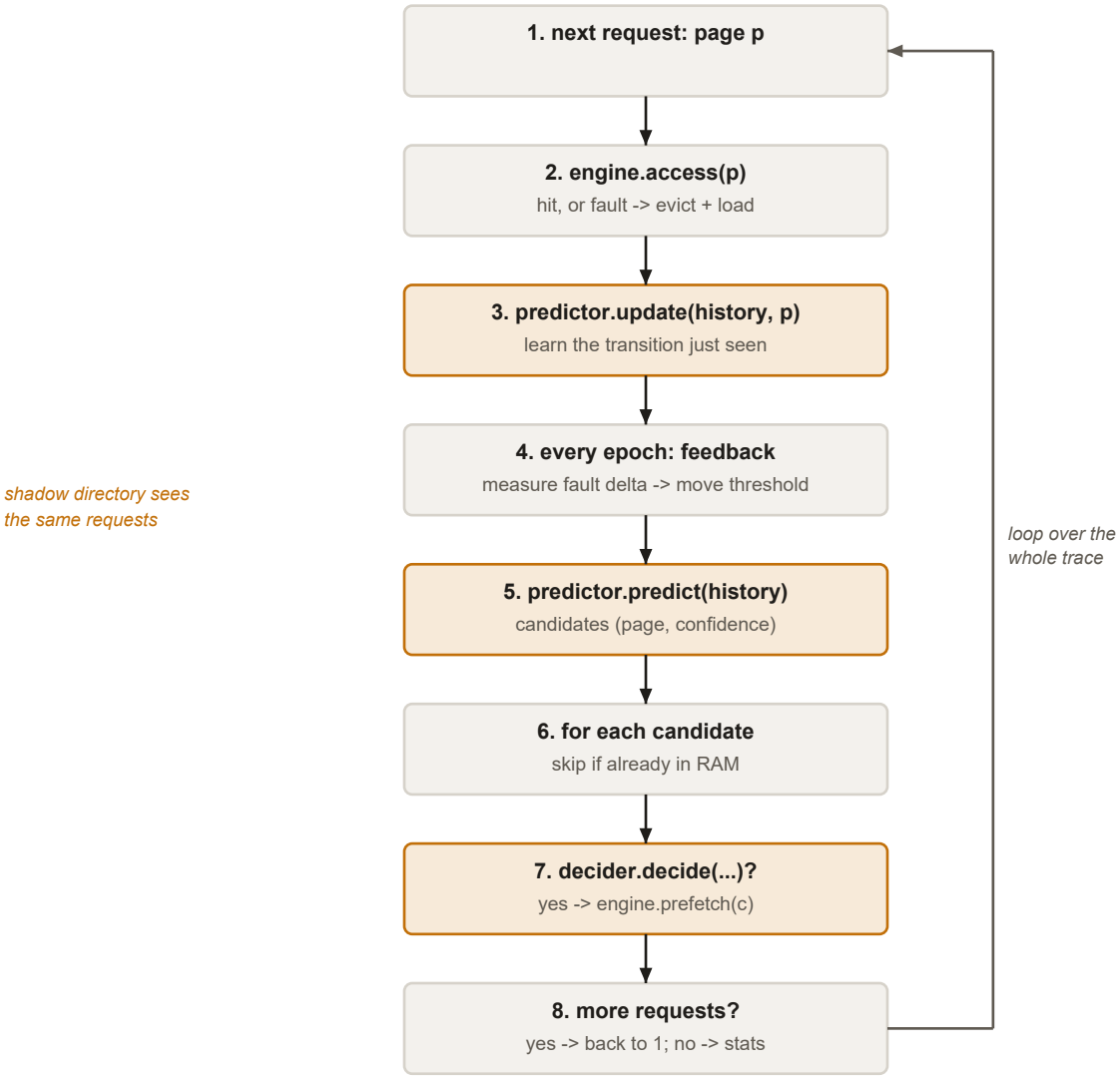
CHAPTER 4

4. Complete working and execution flow

4.1 From start to output

1. **Start.** The user runs a script, e.g. `python scripts/11_robustness.py`. `_path.py` adds `src/` to the import path; `config.py` sets paths (overridable with PAGING_DATA_DIR / PAGING_RESULTS_DIR).
2. **Input.** For each configured trace the script checks that `data/traces/<name>.csv` exists. Missing traces are skipped with a message, so everything still runs on the synthetic traces alone.
3. **Load.** `load_trace` reads the CSV (columns access_id, page_id), keeps the first 200,000 requests, and wraps them in a Trace (page array, gap array, 25% split).
4. **Size memory.** frames = max(8, 1% or 5% of the number of distinct pages).
5. **Build the system.** A policy object (e.g. LRU), a predictor (e.g. the final readahead + delta-Markov), a decider (e.g. the directory decider with frozen settings).
6. **Run the loop** (diagram below) once per request.
7. **Every epoch** (1,000 requests in the final system) the decider receives a fault-delta estimate and moves its threshold.
8. **End.** The engine's counters (faults, evictions, prefetches, used, wasted, precision) and the decider's statistics are returned as a dictionary.
9. **Save.** The script appends one row per configuration to a CSV in `results/` after each trace (so long runs can resume), then prints summary tables.
10. **Present.** `13_report.py` turns all CSVs into `results/report.html`; scripts 18 and 19 build the PDFs.

4.2 The per-request loop



Exactly this order is used in `simulate.run_prefetch_sim`, `run_prefetch_sim_ghost` and `run_prefetch_sim_dir`. Learning happens *before* predicting, and the predictor learns only from requests that have already happened, so it never sees the future.

4.3 What happens in unusual situations

Situation	What the code does	Where
Trace file missing	Configuration skipped; message printed; other traces still run	every script
Download returns an HTML error page	Detected by trying to decompress it; trace skipped as BAD DOWNLOAD	scripts/02
Address field not hexadecimal / short line	Line skipped and counted	traces.convert
Predicted page already in RAM	Candidate skipped before the decider is asked	simulate loops
prefetch() of a resident page	Returns False, nothing changes	MemoryEngine.prefetch
RAM not yet full	No eviction; decider accepts any confidence ≥ 0.15 (a wrong prefetch evicts nothing)	decide()
History too short (fewer pages than the context)	Predictor returns no prediction	MarkovPredictor._key, DeltaMarkov
Same page requested twice in a row	Readahead ignores it (no information); Markov learns A→A	ReadaheadPredictor.update
Ghost decider passed to the plain simulator	ValueError: it would silently run as the oracle	run_prefetch_sim
A long run is interrupted	Rerun the script: finished configurations are read back and skipped	scripts 09, 11, 15, 17
Prefetched page evicted before use	Counted as wasted; the decider is told	MemoryEngine._install

4.4 What the user sees

Printed tables in the terminal, CSV files in `results/`, figures in `results/figures/`, the interactive `results/report.html`, and the PDFs in `docs/`.

CHAPTER 5

5. Complete logic explanation

This chapter explains every important rule in the system: what it does, why, what controls it, and what happens in each case.

5.1 Installing a page (the single entry point)

What and why: every page that enters RAM, by fault or by prefetch, passes through `MemoryEngine._install`. One entry point guarantees correctness and lets prefetched pages be tracked.

```
def _install(self, page, i, is_prefetch=False):
    if page in self.resident:
        return
    if len(self.resident) >= self.n_frames:
        victim = self.policy.choose_victim(self.resident, i)
        self.resident.discard(victim)
        self.policy.on_evict(victim)
        self.n_evict += 1
        if self.prefetched.pop(victim, False):
            self.n_pf_wasted += 1
    self.resident.add(page)
    self.policy.on_insert(page, i)
    if is_prefetch:
        self.prefetched[page] = True
        self.n_prefetch += 1
        mark = getattr(self.policy, 'mark_prefetch', None)    # optional hook (Phase 9)
        if mark is not None:
            mark(page, i)
```

Exact code: src/paging/engine.py, line 32

Case	Result
Page already resident	Nothing happens
RAM has a free frame	Page added; policy told (on_insert)
RAM full	Policy chooses a victim; victim removed; if the victim was an unused prefetch, wasted += 1
Install was a prefetch	Page marked "prefetched, not yet used"; prefetch counter += 1; event P logged if logging

5.2 Handling a request

```
def access(self, page, i):
    self.n_access += 1
    hit = page in self.resident
    if self.prefetched.pop(page, False):
        self.n_pf_used += 1
        if self.log is not None:
            self.log.append(('U', i, page))
    if hit:
        self.policy.on_hit(page, i)
        return False
    self.n_fault += 1
    if self.log is not None:
        self.log.append(('F', i, page))
    self._install(page, i, is_prefetch=False)
    return True
```

Exact code: src/paging/engine.py, line 51

If the page was prefetched and is now requested for the first time, it counts as **used** (and event U is logged for the time model). A hit only updates the policy; a fault counts and installs.

5.3 The decision rule

Inputs: the candidate page, its confidence, and the state (free frames, prefetches so far, requests so far). Output: yes or no.

```
def decide(self, page, conf, state):
    if self.off_left > 0:
        self.d_thr += 1
        return False
    if state['n_prefetch'] > self.budget_ratio * max(1, state['n_access']):
        self.d_budget += 1
        return False
    if state['free_frames'] > 0:
        ok = conf >= self.lo
        if ok: self.n_yes += 1
        else:  self.d_cost += 1
        return ok
    if conf < self.thr:
        self.d_thr += 1
        return False
    self.n_yes += 1
    return True
```

Exact code: src/paging/deciders.py, line 129

Order	Condition	Decision	Why
1	Prefetching switched off (off state, optional)	no	Backing off after repeated losses (never triggered in practice)
2	Prefetches so far > budget × requests	no	Caps prefetch I/O. Final system: budget 1.01 = no cap (Phase 17)
3	Free frames > 0	yes if confidence ≥ 0.15	A wrong prefetch into a free frame evicts nothing, so be generous
4	Confidence < threshold	no	Not sure enough for a full RAM
5	Otherwise	yes	

5.4 The feedback controller

Every epoch the decider receives a **delta rate** = (estimated faults saved in the epoch) / epoch length, and moves its threshold:

```
def _adapt(self, delta_rate):
    self.delta_history.append(round(delta_rate, 5))
    if self.off_left > 0:                       # prefetching is off: wait, then probe at the ceiling
        self.off_left -= 1
        self.n_off_epochs += 1
        self.thr_history.append(round(self.thr, 3))
        return
    if delta_rate > self.dead_band:
        self.thr = max(self.lo, self.thr - self.step)
        self.backoff = 1
    elif delta_rate < -self.dead_band:
        if self.allow_off and self.thr >= self.hi:
            self.off_left = self.backoff
            self.backoff *= 2
        self.thr = min(self.hi, self.thr + self.step)
    self.thr_history.append(round(self.thr, 3))
```

Exact code: src/paging/deciders.py, line 147

Delta rate	Meaning	Action (final settings: step 0.1, dead band 0.002, range 0.15-0.95)
> +0.002	prefetching is saving faults	threshold − 0.1 (bolder), not below 0.15
between −0.002 and +0.002	too small to tell	no change (avoids reacting to noise)
< −0.002	prefetching is causing faults	threshold + 0.1 (more careful), not above 0.95

Why this design: it reacts to the measured outcome, not to an indirect proxy. Stage 5 showed that confidence cannot discriminate (it is never below 0.5) and precision can point the wrong way (Chapter 11).

5.5 Three ways to estimate the fault delta

Estimator	How the "without prefetching" faults are obtained	Cost	Accuracy (measured)
Oracle (CostAwareFaultDelta)	A full second MemoryEngine on the same requests, never prefetching. Uses running totals since the start.	a second copy of memory: impossible in a kernel	exact (by construction)
Ghost list (CostAwareGhost)	Estimate = prefetches used − pages evicted by a prefetch that fault back within 2,000 requests (list of 256 entries)	~4 KB	usually right; biased upward (clustered@9: +988 vs true −1,250)
Shadow directory (CostAwareDirectory)	A second policy instance holding page IDs only, on a hash sample s of pages; estimate = (dir faults − real faults on sampled pages)/s	8 B per tracked frame (~0.2% of RAM); ÷8 with s = 1/8	exact at s = 1; same decisions at s = 1/8 on 9 of 10 traces

```
s = decider.sample
d_frames = max(1, round(n_frames * s))
eng = MemoryEngine(n_frames, policy_cls(pages=trace.pages))
eng.log = log
shadow = MemoryEngine(n_frames, policy_cls(pages=trace.pages))
direc = MemoryEngine(d_frames, policy_cls(pages=trace.pages))
history, prev_wasted, log = [], 0, []
real_s = dir_s = prev_real = prev_shadow = 0

for i, page, _ in trace.stream():
    was_pf = eng.prefetched.get(page, False)
    fault = eng.access(page, i)
    shadow.access(page, i)
    if sampled(page, s):
        real_s += fault
        dir_s += direc.access(page, i)
    if was_pf:
        decider.feedback(page, True)
    if eng.n_pf_wasted > prev_wasted:
        for _ in range(eng.n_pf_wasted - prev_wasted):
            decider.feedback(None, False)
        prev_wasted = eng.n_pf_wasted

    if eng.n_access % decider.epoch == 0:
        est = (dir_s - real_s) / s
        true = (shadow.n_fault - prev_shadow) - (eng.n_fault - prev_real)
        log.append((est, true))
        real_s = dir_s = 0
        prev_real, prev_shadow = eng.n_fault, shadow.n_fault
        decider.directory_update(est)

    history.append(page)
    if len(history) > hist_len:
        history.pop(0)
```

Core of the directory loop: src/paging/simulate.py, line 193

5.6 Predictor logic

Markov update and prediction

```
def update(self, history, actual_next):
    k = self._key(history)
    if k is None:
        return
    if k in self.table:
        self.table.move_to_end(k)
    else:
        if len(self.table) >= self.capacity:
            self.table.popitem(last=False)
            self.evictions += 1
        self.table[k] = {}

    succ = self.table[k]
    succ[actual_next] = succ.get(actual_next, 0) + 1
    if len(succ) > self.max_succ:
        del succ[min(succ, key=succ.get)]

    self.n_updates += 1
    if self.n_updates % self.decay_every == 0:
        self._decay()

def predict(self, history, top_n=1):
    k = self._key(history)
    if k is None or k not in self.table:
        return []
    succ = self.table[k]
    total = sum(succ.values())
    if total == 0:
        return []
    ranked = sorted(succ.items(), key=lambda kv: -kv[1])[:top_n]
    return [(p, c / (total + self.smooth)) for p, c in ranked if c >= self.min_count]
```

Exact code: src/paging/predictors.py, line 33

- **Context table is bounded** (256 entries) and kept in least-recently-used order: a new context evicts the oldest one.
- **Only 2 successors** per context: when a third appears, the least frequent is deleted (often the newcomer itself, if the others have counts ≥ 2: it is then learned only after a decay).
- **Decay:** every 25,000 updates all counts are halved and zero counts removed, so old behaviour fades.
- **Confidence** = count of the best successor / total counts in the context. With ≤ 2 successors this is always ≥ 0.5.

Hybrid chains

HybridPredictor: order-2 Markov → order-1 Markov → stride; the first that answers wins, but *both* Markov tables learn on every request, so a table that is not currently answering is ready after a phase change. **DeltaHybridPredictor** adds the delta-Markov table before stride. **StreamMarkovPredictor** (final) returns the readahead window first, then the delta-hybrid guess if it is not already in the window.

Readahead state machine

```
def update(self, history, actual_next):
    page = int(actual_next)
    self.pending = []
    if self.last is None:
        self.last, self.issued = page, page
        return
    if page == self.last:            # same page again: no information
        return
    if page == self.last + 1:        # sequential continuation
        self.last = page
        self.run += 1
        if self.win == 0:
            if self.run < self.min_run - 1:
                return
            self.win = min(self.init_window, self.max_window)
            self._issue(page, self.win)
        elif page >= self.issued - self.win // 2:
            self.win = min(self.win * 2, self.max_window)
            self._issue(self.issued, self.win)
    else:                            # stream broken
        self.last, self.win, self.issued = page, 0, page
        self.run = 0
        self.n_resets += 1
```

Exact code: src/paging/predictors.py, line 230

Next page	State change
same as last	ignored
last + 1, no stream yet	run += 1; start a stream: window = initial size (4, or a quarter of RAM if smaller); request the next window pages
last + 1, reached the middle of the requested window	double the window (cap: a quarter of RAM) and request the next window beyond what was already requested
anything else	stream broken: window = 0, run = 0

In the final system readahead pages carry confidence `run / (run + 1)` (conf_k = 1), so the decider can block short runs; the original readahead baseline uses 1.0. Note: the docstring of StreamMarkovPredictor still says "confidence 1.0"; that is out of date when conf_k is set.

5.7 Eviction policies in one table

Policy	Bookkeeping	Victim	Note
FIFO	queue of arrivals	oldest arrival still resident	
LRU	OrderedDict ordered by last use	least recently used	main baseline
Clock	ring + reference bits; a hand sweeps	first page with bit 0 (bits cleared on the way)	approximation of LRU
OPT	list of future positions per page	next use furthest away (binary search)	needs the whole trace
ARC	T1 (seen once), T2 (seen twice+), ghost lists B1, B2, target p	from T1 if |T1| > p, else from T2	textbook tie-break dropped
LRU-2	last two use times per page; heap	oldest second-last use (pages seen once first)	no correlated-reference period

CHAPTER 6

6. Source code explanation

6.1 Folder structure

```
predictive-memory-paging/
├── src/paging/            the library
│   ├── config.py          paths, constants, FROZEN predictor settings, CONTROLLER_FROZEN
│   ├── traces.py          synthetic generators, converter, Trace, load_trace
│   ├── predictors.py      Markov, Stride, Hybrid, DeltaMarkov, DeltaHybrid, Readahead, TopN, StreamMarkov
│   ├── engine.py          MemoryEngine, GhostMemoryEngine, FIFO, LRU, Clock, OPTFast, ARC, LRU2, Predictive
    LRU
│   ├── deciders.py        NoPrefetch, AlwaysPrefetch, FixedThreshold, CostAware, CostAwareFaultDelta,
│   │                      CostAwareGhost, CostAwareDirectory
│   ├── simulate.py        eval_predictor, rolling_accuracy, run_prefetch_sim, _ghost, _dir, sampled
│   ├── timing.py          replay (time model), DEVICES
│   └── final.py           make_final_predictor, make_final_decider, run_final
├── scripts/               01 generate · 02 download · 03 predictors · 04 baselines · 05 ablation
│                          06 head-to-head · 07 ghost · 08 harness · 09 Phase 8 · 10 improvements
│                          11 robustness · 12 break-even · 13 report · 14 time model · 15/16 tuning
│                          17 component comparison · 18/19 PDFs · run_all.py
├── tests/test_core.py     invariants and pinned reference numbers
├── docs/                  RESULTS, ROADMAP, DECISIONS_AND_LESSONS, REVIEW_PREP, paper_draft, PDFs
├── data/traces/           trace CSVs (git-ignored)
└── results/               CSVs, figures, report.html (git-ignored except results/reference/)
```

6.2 Responsibilities of the important files

File	Responsibility	Key items
config.py	Single source of settings	TRACE_LIMIT 200,000; WARMUP_FRAC 0.25; FROZEN (capacity 256, max_succ 2, order 2, decay 25,000); CONTROLLER_FROZEN (epoch 1,000, step 0.1, dead band 0.002, budget 1.01); ABLATION_CONFIGS
traces.py	Data in	sequential, strided, cyclic, clustered, uniform_random, phase_changing; convert (addr >> 12); Trace; load_trace; available_traces
engine.py	RAM and eviction	MemoryEngine (_install, access, prefetch, stats, log); GhostMemoryEngine; 6 policies + PredictiveLRU (experimental, not adopted)
predictors.py	Guessing	make_predictor(kind) builds from FROZEN; make_readahead(frames, conf_k)
deciders.py	Acting	decide / feedback / stats interface; _adapt shared controller
simulate.py	Loops	three simulators that differ only in how the fault delta is obtained
timing.py	Time	replay(log, n_access, t_io, channels, t_cpu, t_pred)
final.py	The final system	run_final(trace, frames)
tests/test_core.py	Safety net	24 tests (Chapter 10)

`MemoryEngine` src/paging/engine.py, line 13

Purpose	Simulate RAM: residency, faults, evictions, prefetch bookkeeping.
Inputs	n_frames (RAM size), policy (object with on_insert/on_hit/on_evict/choose_victim).
Logic, step by step	1. access(page, i): count the request; if the page was a pending prefetch, count it used; hit → policy.on_hit; fault → count and _install. 2. prefetch(page, i): ignore if resident; otherwise _install with is_prefetch=True. 3. _install: evict via the policy if full; mark prefetched pages; optional mark_prefetch hook for experimental policies. 4. stats(): faults, fault rate, evictions, prefetches, used, wasted, precision.

Conditions and edge cases	Resident set is a Python set (O(1) membership). The policy owns all ordering. log=None by default (no overhead).
Output / side effects	Counters; resident set; optional event log.
Used by	All three simulators, run_policy, tests.
Worked example	3 frames, requests [1, 2, 3, 1, 4, 1, 2, 5, 1, 2] with LRU: 6 faults (Chapter 9, A).

`LRU` src/paging/engine.py, line 106

Purpose	Evict the least recently used page.
Inputs	Notifications from the engine.
Logic, step by step	1. on_insert / on_hit: store the access index and move the page to the end of an OrderedDict. 2. on_evict: remove it. 3. choose_victim: return the first (oldest) page in the dict that is still resident.
Conditions and edge cases	Insert and hit are O(1); the victim is normally the first entry.
Output / side effects	The victim page.
Used by	Every experiment as the main baseline and underneath prefetching.

```
class LRU:
    def __init__(self, **kw):
        self.od = OrderedDict()

    def on_insert(self, page, i):
        self.od[page] = i
        self.od.move_to_end(page)

    def on_hit(self, page, i):
        self.od[page] = i
        self.od.move_to_end(page)

    def on_evict(self, page):
        self.od.pop(page, None)

    def choose_victim(self, resident, i):
        for p in self.od:
            if p in resident:
                return p
        return next(iter(resident))
```

`OPTFast.choose_victim` src/paging/engine.py, line 180

Purpose	Belady's optimal victim.
Inputs	Resident set and the current position i; precomputed list of positions of every page.
Logic, step by step	1. For each resident page, binary-search its next occurrence after i. 2. Keep the page with the largest next position. 3. Stop early if a page is never used again (infinite distance).
Conditions and edge cases	Needs the full page list at construction (pages=). O(frames × log N) per eviction.
Output / side effects	The victim page.
Used by	Baselines; correctness tests (nothing may beat it).

```
def choose_victim(self, resident, i):
    best, best_d = None, -1
    for p in resident:
        lst = self.occ.get(p, [])
        k = bisect.bisect_right(lst, i)
        d = lst[k] if k < len(lst) else float('inf')
        if d > best_d:
            best, best_d = p, d
            if d == float('inf'):
                break
    return best
```

`MarkovPredictor` src/paging/predictors.py, line 13

Purpose	Learn "context → next page" counts; predict the most frequent next page.
Inputs	capacity, max_succ, order, decay_every (from FROZEN); history list; actual next page.
Logic, step by step	1. _key(history): the last `order` pages as a tuple, or None if history is too short. 2. update: move the context to the most-recent end (or create it, evicting the oldest if full); add 1 to the successor's count; drop the least frequent if more than max_succ; halve all counts every decay_every updates. 3. predict: sort successors by count, return up to top_n with confidence count/total.
Conditions and edge cases	Unknown context → []. All counts zero after decay → []. min_count filters successors below 1.
Output / side effects	List of (page, confidence).
Used by	HybridPredictor (order 1 and 2), DeltaMarkovPredictor (over gaps), scripts 03, 17.
Worked example	After 10,20,30,10,20,30: context (10,) → {20: 2}; predict → [(20, 1.0)] (Chapter 9, B).

`ReadaheadPredictor` src/paging/predictors.py, line 200

Purpose	Linux-style sequential readahead.
Inputs	init_window, max_window (make_readahead: min(4, frames/4) and frames/4), min_run, conf_k; the actual next page.
Logic, step by step	1. Keep last page, run length, window size, highest page already requested, pending list. 2. Sequential step: start or grow the stream (Chapter 5.6 table). 3. Break: reset. 4. predict returns the pending window with confidence 1.0 or run/(run+conf_k).
Conditions and edge cases	Pending is cleared at every update, so a window is offered once. Repeats of the same page are ignored.
Output / side effects	List of (page, confidence) for the whole window.
Used by	Readahead baseline (with AlwaysPrefetch), StreamMarkovPredictor, final system.
Worked example	Chapter 9, C.

`StreamMarkovPredictor` src/paging/predictors.py, line 278

Purpose	The final predictor: readahead window + delta-hybrid Markov guess.
Inputs	n_frames, optional markov and readahead objects.
Logic, step by step	1. update: both parts learn from every request. 2. predict: readahead pending pages first; then the Markov guess if not already listed.
Conditions and edge cases	No new parameters of its own.
Output / side effects	Combined candidate list.
Used by	final.make_final_predictor, scripts 10, 11, 14, 17.

```
def predict(self, history, top_n=1):
    out = list(self.ra.predict(history, top_n))
    have = {p for p, _ in out}
    self.n_ra += len(out)
    for p, c in self.mk.predict(history, top_n):
        if p not in have:
            out.append((p, c))
            self.n_mk += 1
    return out
```

`run_prefetch_sim` src/paging/simulate.py, line 67

Purpose	The basic simulator (no-prefetch, always, fixed, precision and oracle deciders).
Inputs	trace, n_frames, policy_cls, predictor, decider, hist_len=8, top_n=1, log=None.
Logic, step by step	1. Refuse ghost deciders (ValueError). 2. Create the engine (and a shadow engine if the decider has epoch_update: the oracle). 3. For each request: record whether it was a prefetched page; access; shadow access; give feedback (used / wasted). 4. Every epoch: oracle update with running totals. 5. Append the page to the history (last 8); update the predictor with (history[:-1] → page). 6. For each prediction not already resident: build the state; if decide() says yes, prefetch.
Conditions and edge cases	top_n is passed to predict; readahead returns a whole window regardless.
Output / side effects	Engine statistics + decider statistics (+ shadow faults).
Used by	Scripts 05, 06, 09, 10, 14, 17; tests.

```
def run_prefetch_sim(trace, n_frames, policy_cls, predictor, decider,
                     hist_len=8, top_n=1, log=None):
    if hasattr(decider, 'estimate_update'):
        raise ValueError(f'{decider.name} is a ghost-list decider: use run_prefetch_sim_ghost '
                         '(here it would silently run as the shadow-engine oracle)')
    eng = MemoryEngine(n_frames, policy_cls(pages=trace.pages))
    eng.log = log
    history = []
    prev_wasted = 0

    uses_shadow = hasattr(decider, 'epoch_update')
    shadow = MemoryEngine(n_frames, policy_cls(pages=trace.pages)) if uses_shadow else None

    for i, page, _ in trace.stream():
        # 1. demand access (correctness path)
        was_pf = eng.prefetched.get(page, False)
        eng.access(page, i)
        if shadow is not None:
            shadow.access(page, i)
        if was_pf:
            decider.feedback(page, True)

        # 2. report wasted prefetches
        if eng.n_pf_wasted > prev_wasted:
            for _ in range(eng.n_pf_wasted - prev_wasted):
                decider.feedback(None, False)
            prev_wasted = eng.n_pf_wasted

        # 3. fault-delta epoch comparison
        if uses_shadow and eng.n_access % decider.epoch == 0:
            decider.epoch_update(eng.n_fault, shadow.n_fault, eng.n_access)

        # 4. update model
        history.append(page)
        if len(history) > hist_len:
            history.pop(0)
        if len(history) >= 2:
            predictor.update(history[:-1], page)

        # 5. predict + decide
        for cand, conf in predictor.predict(history, top_n=top_n):
            if cand in eng.resident:
                continue
            state = {'free_frames': eng.n_frames - len(eng.resident),
                     'n_frames': eng.n_frames,
                     'n_prefetch': eng.n_prefetch,
                     'n_access': eng.n_access,
                     'n_fault': eng.n_fault}
            if decider.decide(cand, conf, state):
                eng.prefetch(cand, i)
```

`replay (time model)` src/paging/timing.py, line 26

Purpose	Estimate run time from an event log.
Inputs	log of (kind, i, page); n_access; t_io; channels; t_cpu; t_pred.
Logic, step by step	1. For each request i: time += t_cpu + t_pred. 2. U event: if that page's read is not finished, wait until it is (late prefetch). 3. F event: start on the earliest free channel (or now), finish after t_io, wait for it. 4. P event: start on the earliest free channel, do not wait; remember when the page is ready.
Conditions and edge cases	Reads are served in issue order (no priority for demand reads: pessimistic for prefetching). Channels are a min-heap of free times.
Output / side effects	Total time and its parts (fault stall, late stall, compute, predict).
Used by	Script 14, script 17, tests.

Worked example	P at 0, U at 1, t_io 1.0, t_cpu 0.1, 1 channel → late stall 0.9, total 1.1.

```
def replay(log, n_access, t_io, channels, t_cpu, t_pred=0.0):
    free = [0.0] * channels                 # channel free times (min-heap)
    heapq.heapify(free)
    ready = {}                              # page -> time its prefetch read completes
    t = 0.0
    stall_fault = stall_late = 0.0
    k, n = 0, len(log)
    for i in range(n_access):
        t += t_cpu + t_pred
        while k < n and log[k][1] == i:
            kind, _, page = log[k]
            k += 1
            if kind == 'U':                 # recorded at access time, before any fault of the same access
                r = ready.pop(page, 0.0)
                if r > t:
                    stall_late += r - t
                    t = r
            elif kind == 'F':
                start = max(t, heapq.heappop(free))
                done = start + t_io
                heapq.heappush(free, done)
                stall_fault += done - t
                t = done
            else:                           # 'P': issued after this access was served
                start = max(t, heapq.heappop(free))
                done = start + t_io
                heapq.heappush(free, done)
                ready[page] = done
    return {'time': t, 'stall_fault': stall_fault, 'stall_late': stall_late,
            'compute': n_access * t_cpu, 'predict': n_access * t_pred}
```

`convert (trace conversion)` src/paging/traces.py, line 107

Purpose	Turn a raw LoadTrace into access_id,page_id CSV.
Inputs	input file (.xz, .gz or text), output path, address column 2, hex, limit 500,000, page_shift 12.
Logic, step by step	1. Open transparently (lzma / gzip / plain). 2. For each line: split on commas; parse column 2 as hexadecimal; write (n, addr >> 12). 3. Skip short or unparsable lines; stop at the limit.
Conditions and edge cases	Bad lines are counted, not fatal. Box downloads are validated earlier by trying lzma.open.
Output / side effects	(rows written, rows skipped).
Used by	scripts/02.

`run_final` src/paging/final.py, line 23

Purpose	One call for the final system.
Inputs	trace, n_frames, policy (default LRU), log, sample.
Logic, step by step	1. Build StreamMarkovPredictor(readahead conf_k = 1, delta-hybrid Markov). 2. Build CostAwareDirectory(**CONTROLLER_FROZEN). 3. Run run_prefetch_sim_dir.
Conditions and edge cases	Settings are frozen; do not retune on evaluation traces.
Output / side effects	Statistics dict.
Used by	Scripts 11, 12, 14, 17; tests.

```
from .config import CONTROLLER_FROZEN
from .deciders import CostAwareDirectory
from .engine import LRU
from .predictors import make_predictor, make_readahead, StreamMarkovPredictor
from .simulate import run_prefetch_sim_dir


def make_final_predictor(n_frames):
    return StreamMarkovPredictor(n_frames, markov=make_predictor('delta-hybrid'),
                                 readahead=make_readahead(n_frames, conf_k=1))


def make_final_decider(sample=1.0):
    return CostAwareDirectory(sample=sample, **CONTROLLER_FROZEN)


def run_final(trace, n_frames, policy_cls=LRU, log=None, sample=1.0):
    return run_prefetch_sim_dir(trace, n_frames, policy_cls, make_final_predictor(n_frames),
                                make_final_decider(sample), log=log)
```

6.3 How the scripts use the library

Script	What it does	Main output
01, 02	generate synthetic traces; download + convert real traces (urllib, validated)	data/traces/*.csv
03	predictor quality, attribution, capacity sweep	predictor_results.csv
04	FIFO/LRU/Clock/OPT across memory sizes with correctness checks	baseline_results.csv
05, 06, 07	decider ablation; head-to-head; ghost list vs oracle	ablation_results.csv, head_to_head.csv, ghost_estimate_results.csv
08	Phase 6 harness (segments, sweep, seeds)	phase6_*.csv
09, 10	ARC/LRU-2/readahead; improvement experiments	phase8_*.csv, improvements.csv
11, 12, 14	robustness on 22 traces; break-even cost; time model	robustness.csv, breakeven.csv, time_model.csv
15, 16	controller and budget tuning (warm-up only)	tuning_*.csv
17	every predictor and decider side by side	component_*.csv
13, 18, 19	HTML report, explainer PDF, this guide	report.html, docs/*.pdf

CHAPTER 7

7. Algorithms and mathematical logic

Everything in this chapter is **implemented** unless marked otherwise. Section 7.4 lists what is only discussed.

7.1 Formulas

Quantity	Formula	Variables	Worked example
Page number	page = address >> 12	address in bytes	0x7f3a2c18 >> 12 = 0x7f3a2
Coverage, accuracy, useful	coverage = predictions / requests; accuracy = correct / predictions; useful = coverage × accuracy	top-1 guess vs actual next page	coverage 0.98, accuracy 0.86 → useful 0.84 (leslie3d hybrid)
Markov confidence	conf = count(best) / Σ counts in the context	counts after decay	{30: 2, 40: 1} → 2/3 = 0.67
vs_LRU	(faults_LRU − faults) / faults_LRU	same trace and frames	(15,313 − 9,956) / 15,313 = 0.35
Headroom	(faults_LRU − faults_OPT) / faults_LRU		(15,313 − 5,881) / 15,313 = 0.62
Precision	prefetches used / prefetches issued		5,486 / 5,841 = 0.94 (leslie3d, always)
Eviction inflation	evictions with prefetching / without		1.93 on clustered@6 (always)
Delta rate	(faults_without − faults_with) / epoch	per epoch	(130 − 100) / 1,000 = 0.03
Ghost estimate	used − harmful refaults	refault within 2,000 requests	illustrative: used 40, harm 55 → −15 (losing)
Directory estimate	(dir faults − real faults on sampled pages) / s	s = sampling rate	illustrative: (14 − 9) / 0.125 = 40
Hash sample	$(page \times 2654435761 \bmod 2^{32}) < s \times 2^{32}$	Knuth multiplicative hash	s = 1/8: pages [5, 13, 18, 26] ... of 1-40
Readahead confidence	run / (run + 1)	run = consecutive +1 steps	run 1 → 0.5; run 9 → 0.9
Break-even cost	c* = (faults_LRU − faults) / prefetches	cost of a prefetch as a fraction of a fault	(15,313 − 9,956) / 5,839 = 0.92
Speed-up	time_LRU / time_system	from the time model	1.43 = 43% faster
Geometric mean	exp(mean(log x))	averages ratios fairly	gm(2, 0.5) = 1.0

7.2 Complexity and memory

Operation	Time per call	Memory
Residency check, LRU update	O(1)	one entry per resident page
FIFO / LRU victim	O(1) normally (scans stale entries rarely)	
Clock eviction	O(frames) worst case (list removal)	ring of frames
OPT victim	O(frames × log N)	positions of every request (N)
LRU-2 victim	O(log n) with lazy-deletion heap	history for resident + retained pages
Markov update/predict	O(1) (≤ 2 successors)	256 contexts per table: order-2 8.7 KB, order-1 6.7 KB, delta ~8.7 KB (arithmetic, kernel-style accounting)
Ghost list	O(1)	256 × 16 B ≈ 4 KB
Shadow directory	same as the policy	8 B per tracked frame; s × frames frames
Time-model replay	O(events × log channels)	one entry per pending prefetch

7.3 Assumptions and limits of the maths

- vs_LRU and speed-up compare one run of each system on the same deterministic trace: no randomness, so no error bars, but also no evidence about other inputs.
- Break-even c* assumes a prefetch costs a fixed fraction of a fault; the time model replaces that with explicit latencies but those are stated typical values, not measurements.
- Confidence is not a calibrated probability (structural floor of 0.5).
- The fault delta per epoch is noisy when the effect is tiny; the dead band ignores |rate| ≤ 0.002.

7.4 Implemented vs only discussed

Implemented and measured	Implemented, tried, not adopted	Only discussed (not implemented)
FIFO, LRU, Clock, OPT, ARC, LRU-2; stride, Markov o1/o2, hybrid, delta-Markov, readahead, combination; 9 deciders; ghost list; shadow directory with sampling; time model; break-even	PredictiveLRU (stale-prefetch eviction, predictor protection); top-2 prefetching; smoothed confidence; longer minimum stream; off state with backoff; 35% budget cap (removed)	LSTM / attention predictors; PACMan as a separate baseline; pointer-chasing prefetchers (Tyche); measured hardware timing; OS-level fault traces

CHAPTER 8

8. Technologies, tools and dependencies

Technology	What it is	Role in the project	Essential?
Python 3.9+	Programming language	All code; chosen for fast iteration (the hot loop is dict and set operations)	Yes (a C++ port would only matter for speed claims)
NumPy	Numerical arrays	Trace page arrays, gaps, statistics, geometric means	Yes
pandas	Tables	Read traces, write and analyse result CSVs	Yes (replaceable by csv + loops)
matplotlib	Plotting	Figures (monochrome + amber)	Only for figures
pytest	Test runner	24 invariant and regression tests	Yes for development
reportlab	PDF generation	This guide and the explainer PDF	Only for PDFs
lzma, gzip, csv (stdlib)	Decompression, CSV	Read .txt.xz LoadTraces, convert to CSV	Yes
urllib (stdlib)	HTTP download	Download traces from Box (replaced wget, absent on Windows)	Replaceable
collections, heapq, bisect (stdlib)	Data structures	OrderedDict (LRU, Markov tables, ARC), heaps (LRU-2, disk channels), binary search (OPT)	Yes
ML-DPC LoadTraces / ChampSim links	Public trace collection	Real program recordings (SPEC CPU 2006, GAP)	Yes for real results
Google Colab + Drive	Hosted notebooks	Where Stages 1-5 were first run	No (local runs reproduce them)
Git, VS Code	Version control, editor	Development	No

Installation: `pip install -r requirements.txt -r requirements-dev.txt`. Traces need `git clone https://github.com/Quangmire/ChampSim.git` for the download links, then script 02. On Windows, if `python` is not found, use `py`.

CHAPTER 9

9. Data flow and worked examples

Every table in this chapter was produced by running the project's own classes on the input shown (inside `scripts/19_study_guide_pdf.py`).

Example A: three eviction policies, 3 frames

Input: requests [1, 2, 3, 1, 4, 1, 2, 5, 1, 2], RAM of 3 frames, no prefetching. F = fault; the set is RAM after the request; "out" is the evicted page.

Req	FIFO	LRU	OPT
1	**F** [1]	**F** [1]	**F** [1]
2	**F** [1, 2]	**F** [1, 2]	**F** [1, 2]
3	**F** [1, 2, 3]	**F** [1, 2, 3]	**F** [1, 2, 3]
1	hit [1, 2, 3]	hit [1, 2, 3]	hit [1, 2, 3]
4	**F** [2, 3, 4] (out 1)	**F** [1, 3, 4] (out 2)	**F** [1, 2, 4] (out 3)
1	**F** [1, 3, 4] (out 2)	hit [1, 3, 4]	hit [1, 2, 4]
2	**F** [1, 2, 4] (out 3)	**F** [1, 2, 4] (out 3)	hit [1, 2, 4]
5	**F** [1, 2, 5] (out 4)	**F** [1, 2, 5] (out 4)	**F** [1, 2, 5] (out 4)
1	hit [1, 2, 5]	hit [1, 2, 5]	hit [1, 2, 5]
2	hit [1, 2, 5]	hit [1, 2, 5]	hit [1, 2, 5]
faults	7	6	5

Why: at request 5 (page 4) FIFO evicts 1, the oldest arrival, although 1 is used next; LRU evicts 2, the least recently used; OPT evicts 3, never used again. OPT wins because it sees the future.

Example B: Markov order-1 learning a pattern

Input: a repeating pattern 10, 20, 30 with one irregular step (40). The prediction is made from the history *before* the request arrives.

Request	Prediction made before it	Correct?
10	none (context unknown)	
20	none (context unknown)	
30	none (context unknown)	
10	none (context unknown)	
20	20 (conf 1.00)	yes
30	30 (conf 1.00)	yes
10	10 (conf 1.00)	yes
20	20 (conf 1.00)	yes
40	30 (conf 1.00)	**no**
10	none (context unknown)	
20	20 (conf 1.00)	yes
30	30 (conf 0.67)	yes

Final table (context → successor counts): {30: {10: 2}, 40: {10: 1}, 10: {20: 4}, 20: {30: 3, 40: 1}}. The irregular 40 was mispredicted (the table said 30), and the request after it had no prediction because context 40 had never been seen. For the last request, context 20 held the counts 30: 2 and 40: 1, so the prediction 30 had confidence 2 / (2 + 1) = 0.67; after that request the count of 30 became 3.

Example C: readahead window growth

Input: a run 100-108, then a jump to 200. ReadaheadPredictor(initial window 2, maximum 8).

Request	run	window	highest requested	pages fetched now
100	0	0	100	-
101	1	2	103	102, 103
102	2	4	107	104, 105, 106, 107
103	3	4	107	-
104	4	4	107	-
105	5	8	115	108, 109, 110, 111, 112, 113, 114, 115
106	6	8	115	-
107	7	8	115	-
108	8	8	115	-
200	0	0	200	-
201	1	2	203	202, 203
202	2	4	207	204, 205, 206, 207

Why: 100, 101 starts a stream (2 pages ahead). At 102 the program reaches the middle of the requested window (103 − 2/2), so the next window doubles to 4; at 105 (107 − 4/2) it doubles to 8. The jump to 200 resets everything.

Example D: delta-Markov predicts pages never seen

Request	Gap	Delta-Markov predicted
1000		-
1001	+1	-
1004	+3	-
1005	+1	-
1008	+3	-
1009	+1	1009
1012	+3	1012
1013	+1	1013
1016	+3	1016

After seeing the gap pattern +1, +3 twice, it predicts the next page correctly although 1009, 1012, 1013 and 1016 had never appeared: a page-number Markov table could not.

Example E: a full prefetching run, step by step

Input: the loop 1, 2, 3, 4, 5 repeated 3 times; RAM of 4 frames; LRU underneath; hybrid Markov predictor; "always" decider. Without prefetching LRU faults on **every** request (15 faults): the loop is one page bigger than RAM, so LRU always evicts the page needed next. With prefetching:

#	Page	Result	Evicted by fault	Prefetched (conf)	Evicted by prefetch	RAM after
0	1	**FAULT**	-	-	-	[1]
1	2	**FAULT**	-	-	-	[1, 2]
2	3	**FAULT**	-	-	-	[1, 2, 3]
3	4	**FAULT**	-	5 (1.00)	1	[2, 3, 4, 5]
4	5	hit	-	6 (1.00)	2	[3, 4, 5, 6]
5	1	**FAULT**	3	2 (1.00)	4	[1, 2, 5, 6]
6	2	hit	-	3 (1.00)	5	[1, 2, 3, 6]
7	3	hit	-	4 (1.00)	6	[1, 2, 3, 4]
8	4	hit	-	5 (1.00)	1	[2, 3, 4, 5]
9	5	hit	-	1 (1.00)	2	[1, 3, 4, 5]
10	1	hit	-	2 (1.00)	3	[1, 2, 4, 5]
11	2	hit	-	3 (1.00)	4	[1, 2, 3, 5]
12	3	hit	-	4 (1.00)	5	[1, 2, 3, 4]
13	4	hit	-	5 (1.00)	1	[2, 3, 4, 5]
14	5	hit	-	1 (1.00)	2	[1, 3, 4, 5]

Result: 5 faults instead of 15. Step by step: at #3 the stride part sees gaps +1, +1, +1 and prefetches 5 (correct: #4 is a hit). At #4 it predicts 6, which does not exist in the loop: a **wrong guess** that evicts page 2 (2 is brought back later by a correct prefetch, so this guess wastes a read but causes no extra fault). At #5 page 1 faults because it was evicted at #3. From #5 on the order-1 Markov table knows the loop and every request is a hit.

Example F: the controller over five epochs

Final settings (epoch 1,000, step 0.1, dead band 0.002). The "no-prefetch faults" come from the shadow directory.

Epoch	Real faults	No-prefetch faults	Delta rate	Threshold before → after	Reason
1	100	130	+0.030	0.50 → 0.40	helping → bolder
2	110	125	+0.015	0.40 → 0.30	helping → bolder
3	120	121	+0.001	0.30 → 0.30	inside dead band → unchanged
4	140	120	-0.020	0.30 → 0.40	hurting → careful
5	150	120	-0.030	0.40 → 0.50	hurting → careful

Example G: the time model

Event log	Settings	Result	Explanation
P(7) at request 0, U(7) at request 1	t_io 1.0, t_cpu 0.1, 1 channel	time 1.1, late stall 0.9	The read finishes at 1.1; the page is needed at 0.2, so the program waits 0.9
P(7) at request 0, F(8) at request 1	same	time 2.1	The fault must wait for the prefetch read on the single channel, then take its own 1.0

CHAPTER 10

10. Testing, evaluation and results

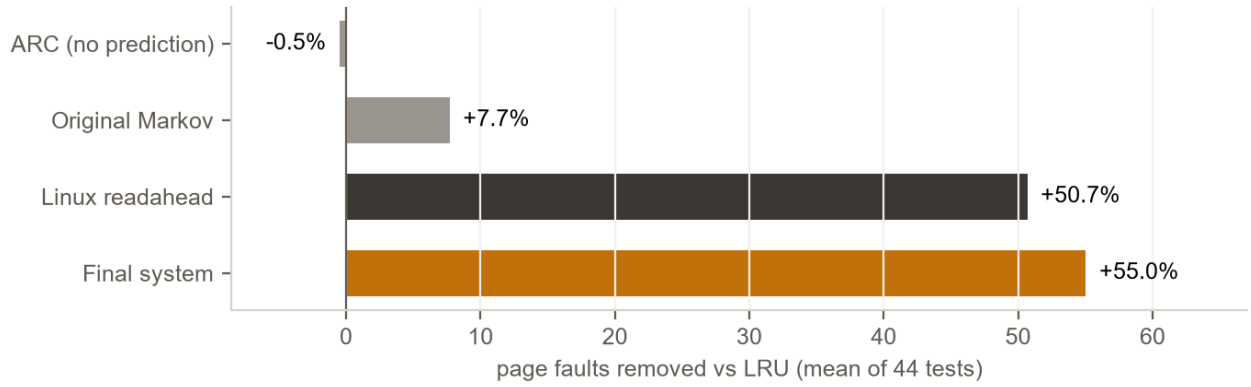
10.1 How the project is tested

`tests/test_core.py` contains 24 tests (run with `python -m pytest -q`, about 15-60 s). They fall into four groups:

Group	Examples	Type of scenario
Invariants	OPT is a lower bound for every policy; LRU and OPT never get worse with more memory; with ample memory faults = distinct pages; a sequential scan faults every time	normal + boundary
Equivalence ("old behaviour reproduced")	ghost engine = plain engine without prefetching; PredictiveLRU off = LRU; allow_off off = old decider; readahead min_run default = original	regression
Pinned numbers	clustered@9 FIFO 5,729, Clock 5,676, LRU 5,803, OPT 2,637, ghost 5,798; LRU-2 6,512; ARC 6,207; directory 5,948; final 5,942 and 20,522	regression (any change means behaviour changed)
Component checks	ARC ghost lists stay bounded; stride needs three equal gaps; delta-Markov predicts unseen pages; full directory = true delta every epoch; time model charges late prefetches and queueing; ghost decider refused by the plain simulator	normal + invalid input

Beyond the tests: synthetic traces with known answers, an independent re-implementation of LRU and FIFO and OPT (0 mismatches on 40 random traces) and of LRU-2 (identical on three real traces), and a full rerun on freshly downloaded traces that reproduced all 48 rows of the original Colab ablation exactly.

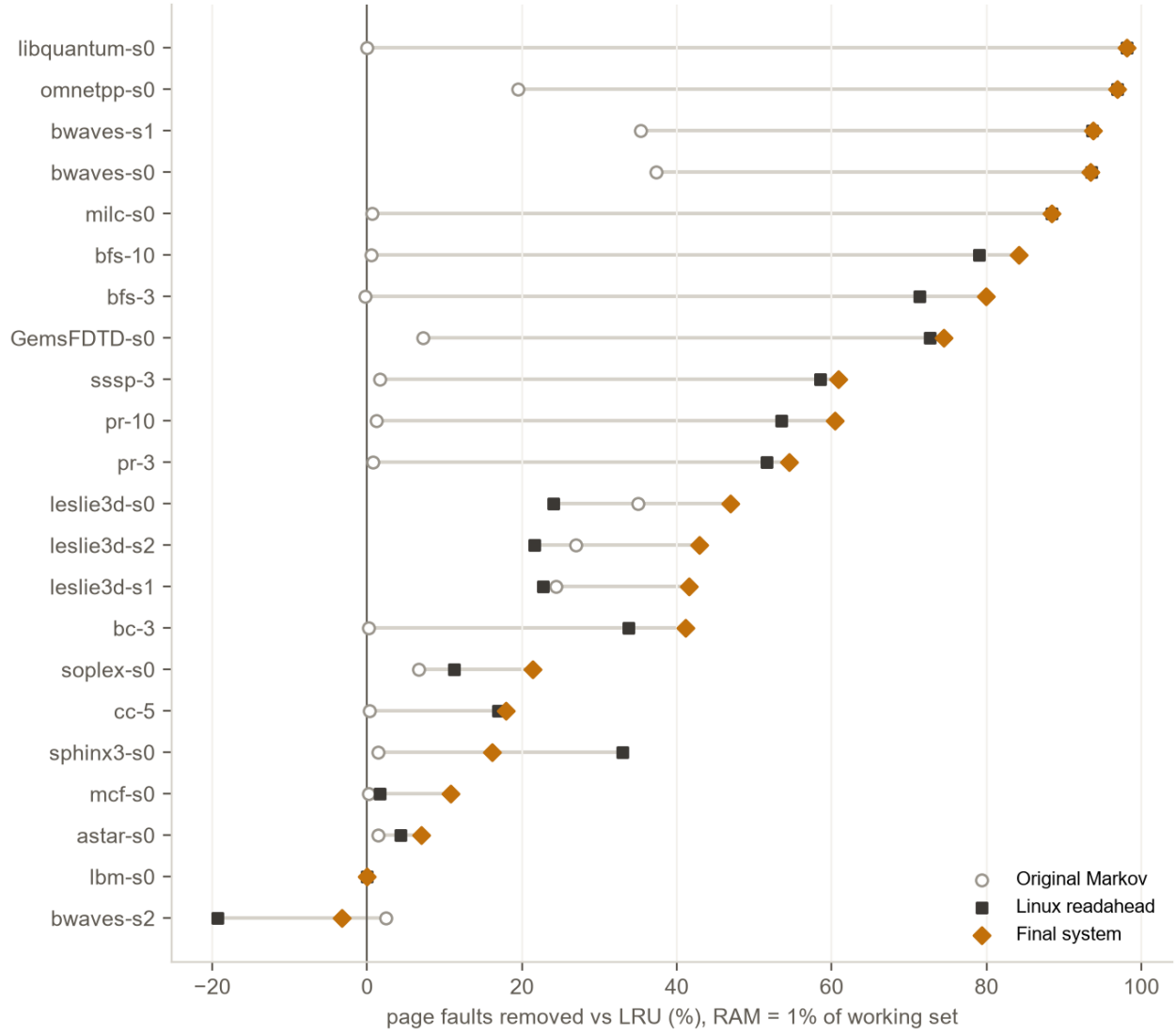
10.2 Results



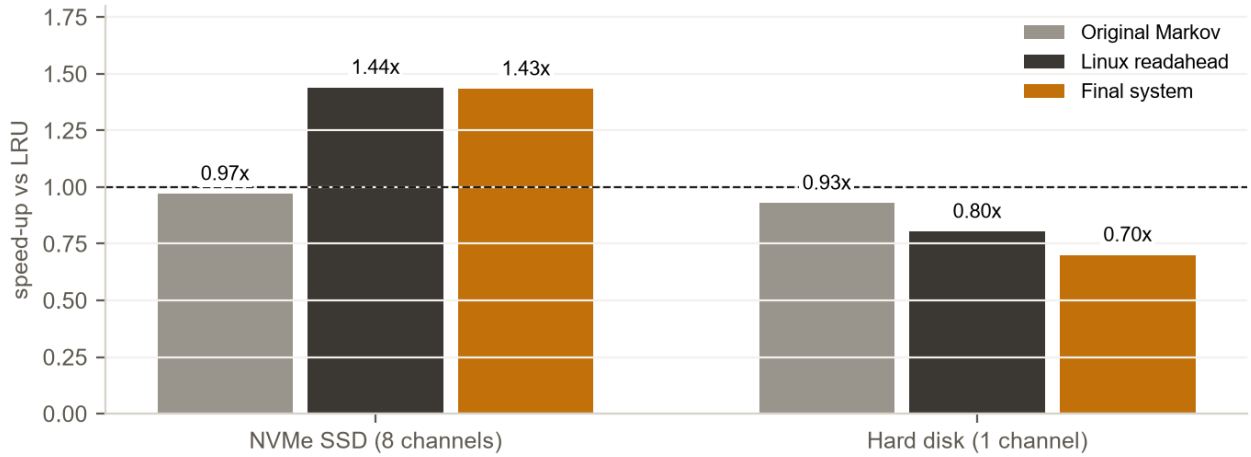
Mean share of LRU's faults removed, 22 real programs × 2 memory sizes. Source: results/robustness.csv (script 11).

System	Mean vs LRU	Worst case	Worse than LRU
ARC (eviction only)	-0.5%	-8.2%	22/44
Original Markov + ghost decider	+7.7%	-0.3%	1/44
Linux-style readahead	+50.7%	-19.3%	2/44
Final system	**+55.0%**	**-3.3%**	**1/44**

Final vs readahead: fewer faults in 34, equal in 8, more in 2 of 44. Below OPT in 30 of 44 (prefetching removes compulsory faults).



Per program at 1% memory. lbm has no predictable pattern; sphinx3 is where the decider holds back useful readahead.



Time model, 22 programs, 1 µs compute per request, 0.2 µs prediction cost. Source: results/time_model.csv (script 14).

System	NVMe SSD (8 channels)	Hard disk (1 channel)
Original Markov	0.97x (slower on 18/22)	0.93x
Readahead	1.44x (slower on 2/22)	0.80x
Final system	**1.43x** (slower on 3/22)	**0.70x**

10.3 Every component side by side

One component changed at a time, every real program at 1% memory (script 17).

Predictor (same final decider)	Exact next-page guesses	Faults removed (mean)	Worse than LRU	SSD speed-up
stride	3%	+5.8%	0	1.01x
markov-o1	50%	+4.9%	3	1.01x
markov-o2	53%	+2.4%	3	1.00x
hybrid	57%	+9.4%	1	1.01x
delta-hybrid	59%	+20.9%	0	1.02x
readahead	3%	+46.5%	1	1.51x
final (readahead + delta-hybrid)	56%	+51.2%	1	1.49x

Decider (fed the final predictor)	Mean	Worst real	Clustered (synthetic)	SSD
no-prefetch	+0.0%	+0.0%	+0.0%	1.00x
always	+51.2%	-6.3%	-21.5%	1.49x
fixed-0.3	+51.2%	-6.3%	-21.5%	1.49x
fixed-0.6	+28.8%	+0.0%	-5.5%	1.21x
fixed-0.85	+19.8%	+0.0%	-0.2%	1.07x
precision (cost-aware)	+26.8%	+0.0%	-5.2%	1.16x
fault-delta oracle	+49.1%	+0.0%	-1.7%	1.39x
ghost list	+47.9%	-3.2%	-21.5%	1.39x
shadow directory	+49.0%	-1.4%	-2.5%	1.39x
directory + frozen controller (final)	+51.2%	-3.3%	-2.4%	1.49x

Prediction accuracy and fault reduction rank predictors almost in opposite orders. "Always" gets the full benefit but is badly hurt on the synthetic clustered trace; fixed thresholds are safe but lose half the benefit; the final decider keeps the benefit and limits the harm.

10.4 What the results show and do not show

Shown (measured)	Not shown
Fault reductions on 22 real LLC traces at two memory sizes, every number reproducible	Behaviour on real OS page-fault traces or real hardware
Which feedback signal works under eviction pressure	Statistical spread from repeated runs (traces are deterministic; robustness comes from more traces)
Where fewer faults become speed-ups in a stated time model	Measured run time; demand-read priority, write-back, I/O merging
Shadow directory sampling at 1/8 on memories of 13-1,163 frames	Sampling at kernel-scale memory (millions of frames)

10.5 Known issues and unverified claims

- The predictor sees every memory request; a kernel sees only faults and accessed bits, so the information is optimistic and the prediction cost charged is an upper bound.
- StreamMarkovPredictor's docstring says readahead confidence is 1.0; with conf_k set it is run/(run+1).
- The ghost list's per-epoch estimate is biased upward on some workloads (it counts every used prefetch as a saved fault).
- The memory footprint figures are arithmetic from the data structures (kernel-style accounting), not measured; Python objects are much larger.
- Controller and budget settings were tuned on warm-up data dominated, for the graph traces, by a sequential initialisation phase, which could not inform any choice there.

CHAPTER 11

11. Novelty and design decisions

11.1 What is genuinely ours and what is borrowed

Borrowed (cite it)	Our contribution
Markov prefetching (Joseph and Grunwald, ISCA 1997)	Evaluation at page granularity of which feedback signal a cost-aware prefetcher should use (confidence and precision fail; fault delta works)
Fault-rate feedback (Page Fault Frequency)	Showing that a cheap ghost-list estimate is biased in the same way precision is, and replacing it with an exact ID-only shadow directory, samplable (SHARDS-style) without changing decisions
Linux readahead; ghost lists (ARC, Linux workingset); SHARDS sampling	Combining readahead with a delta-Markov predictor and quantifying where each helps on 22 traces
Prefetch-aware insertion (PACMan, MICRO 2011)	A time model showing fewer faults ≠ faster: timeliness, extra reads and device parallelism decide; plus documented negative results

Do not claim "a new algorithm". Claim "an evaluation study plus a deployable feedback mechanism".

11.2 Design decisions, alternatives and evidence

Decision	Alternative	Why this one (evidence)
Trace-driven simulation	Kernel implementation	Identical input for every policy, OPT possible, far less engineering
Markov tables, not deep learning	LSTM / attention	Runs on every request: one lookup, KB of state; a learned model is future work
Memory as % of working set	Fixed frame counts	With 64-512 frames bwaves gave identical results under every policy
Prediction on top of LRU	Replace LRU	Gain was the same on FIFO, Clock, LRU (34.5-38.6%)
Fault delta as feedback	Precision, confidence	Precision 0.81-0.86 with rising faults; confidence never below 0.5
Shadow directory	Ghost list	clustered@9: −21.5% → −2.5%; best worst case over 26 points
Readahead + delta-Markov	Markov alone	Readahead beat Markov on 3 of 4 traces; the combination beats both on most
No budget cap	35% cap	Cap blocked omnetpp while winning every epoch; no warm-up trace got worse
Warm-up-only tuning, rule fixed in advance	Tune on full traces	Avoids reporting numbers tuned on the test data

11.3 Trade-offs

- More prefetching removes more faults but costs more reads: the final system ties readahead in time and loses on a hard disk.
- A cautious decider protects the worst case but can hold back useful prefetches (sphinx3).
- The shadow directory is exact but costs memory proportional to the sample; sampling trades estimate noise for size.

CHAPTER 12

12. Limitations and future improvements

12.1 Current limitations (existing system)

Kind	Limitation
Data	LLC traces treated as pages; 200,000 requests per program; 22 programs
Model	Predictor sees every request; no demand-read priority, write-back or I/O merging; stated latencies
Scale	Simulated memories of 8-1,163 frames; Python speed (seconds per configuration)
Accuracy	One run per configuration; graph warm-ups uninformative for tuning
Baselines	ARC without the textbook tie-break; LRU-2 without correlated-reference period; readahead window = RAM/4
Reliability	Simulator only: no kernel safety, concurrency or security aspects are modelled

12.2 Suggested improvements (not implemented)

Improvement	How it would help
OS-level page-fault traces (e.g. from a kernel tracer)	Removes the LLC approximation and the "sees every request" optimism
Demand-read priority and prefetch distance control in the time model	More realistic SSD results; addresses late prefetches
Charge indirect harm in the ghost estimate	Could make the cheap estimate unbiased
Learned predictor under the same time model	Fair comparison with recent ML prefetchers
Sampling at realistic memory sizes	Validates the directory's deployability claim
Per-workload decider for sphinx3-like cases	Recover gains the cautious decider leaves behind

CHAPTER 13

13. Important questions and answers

32 questions grouped by topic. Answers use only measured facts from this project.

Topic	Question	Answer
Basic	**What is a page fault?**	A request for a page that is not in RAM; the program waits while it is read from disk.
Basic	**What is prefetching and why is it risky?**	Loading a page before it is requested. With full RAM each prefetch evicts a page; a wrong guess can evict a needed page and create a fault.
Basic	**Why compare with LRU?**	It is the standard eviction policy and what real systems approximate (Clock). All vs_LRU numbers use it.
Basic	**What is OPT and how can you beat it?**	Belady's optimal eviction (needs the future). It still pays every first-touch fault; prefetching avoids those, so prefetching can go below it.
Basic	**What is a trace?**	A recording of the memory requests of a real program; we replay it through every method.
Architecture	**Why separate predictor and decider?**	So 9 deciders can be compared on byte-identical predictions, and any difference comes from the decider.
Architecture	**How do prefetched pages enter RAM?**	Only through MemoryEngine._install, the same path as demand faults, so a bad predictor can slow but never corrupt the system.
Architecture	**What does the shadow directory store?**	Page IDs only, in a no-prefetch copy of the eviction policy: about 8 bytes per tracked frame.
Architecture	**What is the final system?**	Readahead (confidence run/(run+1)) + delta-hybrid Markov predictor, directory-fed fault-delta decider with frozen settings, LRU underneath (paging.final.run_final).
Algorithms	**How does the Markov predictor learn?**	It counts "context → next page" in a 256-entry table, keeps the two most frequent successors, halves counts every 25,000 updates.
Algorithms	**Why is confidence never below 0.5?**	Only two successors are kept, so the top one always has at least half the counts. It is structural, not a property of the programs.
Algorithms	**What does delta-Markov add?**	It learns gaps, so it predicts pages never seen before (bwaves: Markov 1,969 → 810 faults).
Algorithms	**How does readahead decide the window?**	Two consecutive pages start a stream; reaching the middle of the requested window doubles it, up to a quarter of RAM; any jump resets.
Algorithms	**How does the controller move the threshold?**	Every epoch: delta rate > 0.002 → threshold − step; < −0.002 → + step; otherwise unchanged. Final: epoch 1,000, step 0.1.
Algorithms	**Why did precision fail as a signal?**	A used prefetch can still have evicted a page needed sooner; on clustered@9 precision was 0.81-0.86 while faults rose.
Code	**Why does run_prefetch_sim refuse the ghost decider?**	The ghost decider inherits epoch_update, so the plain simulator would silently run it as the oracle.
Code	**Why OrderedDict for LRU?**	move_to_end and pop-first are O(1), giving LRU order cheaply.
Code	**How is OPT computed efficiently?**	Precomputed positions of every page; binary search for the next use of each resident page.
Code	**How are long runs made safe to interrupt?**	Scripts save a CSV row per configuration and skip finished ones on restart.

Topic	Question	Answer
Testing	**How do you know the simulator is correct?**	24 tests (invariants, equivalences, pinned numbers), independent re-implementations, and an exact reproduction of the Colab results.
Testing	**Why no error bars?**	The traces are deterministic, so reruns are identical. Robustness comes from 22 programs, 3 segments of two programs and 2 memory sizes.
Testing	**How were settings chosen without cheating?**	Only on the first 25% of each trace, with the selection rule written down before the sweep.
Results	**What is the headline?**	55.0% fewer faults than LRU on average over 44 tests, worse than LRU once; in time 1.43x on an SSD model (readahead 1.44x), slower on a hard disk.
Results	**Why is Markov alone not faster although it removes 35% of faults on leslie3d?**	It prefetches one request ahead, so with an 80 µs read the page arrives late; 16-22% of run time was waiting for late prefetches.
Results	**Why does prefetching lose on a hard disk?**	One read at a time: every prefetch costs as much as the fault it tries to save, and every break-even cost is below 1.
Limitations	**Is this OS paging?**	An approximation: LLC traces at page granularity, and the predictor sees every request, which a kernel does not.
Novelty	**What is new?**	The evaluation of feedback signals, the exact samplable shadow directory as the feedback, the readahead + delta-Markov combination and the time-based evaluation; not the building blocks.
Challenging	**If readahead ties you in time, why not just use readahead?**	In time, on these traces, readahead is as good; our system removes more faults (34 of 44 points), helps much more on leslie3d-like workloads and limits harm better than readahead's −19% worst case.
Challenging	**Is the shadow directory just the oracle again?**	Same idea, but it stores IDs only (no data) and can be sampled to 1/8; the earlier claim that a kernel cannot run any shadow was too strong.
Challenging	**What would make your results wrong?**	A different access stream (OS-level faults), larger memories, real latencies with demand priority, or tuning that leaked test data. We guarded the last one; the others are future work.
Challenging	**Why did ARC not help?**	On these traces ARC was worse than LRU on leslie3d at 1% (16,553 vs 15,313) and about equal elsewhere; its strength (scan resistance) is not what these workloads need.
Challenging	**What happens to clustered@9 and why?**	Random hops inside a 10-page cluster look like short runs; the final system is 2.4% worse than LRU there, "always" 21.5% worse; the directory decider limits the damage.

CHAPTER 14

14. Quick revision

14.1 One-page summary

Problem: RAM is small, disks are slow; page faults make programs wait. **Idea:** predict the next pages and prefetch them, but only when it helps. **System:** a trace-driven simulator with a memory engine (6 eviction policies), 6 predictors, 9 decision engines and a time model. **Final system:** readahead + delta-Markov predictor; a decider that moves a confidence threshold using the exact fault delta from a page-ID-only shadow directory; LRU underneath. **Results:** 55.0% fewer faults than LRU on 22 real programs (readahead 50.7%, Markov alone 7.7%), worse than LRU in 1 of 44; 1.43x faster than LRU on an SSD model (readahead 1.44x), slower on a hard disk. **Lessons:** use the outcome as feedback; accuracy ≠ fewer faults ≠ faster; timeliness and device parallelism decide.

14.2 Working flow in numbered steps

1. Load a trace (page numbers) and size RAM as 1% or 5% of its distinct pages.
2. For each request: the engine reports hit or fault; on a fault the policy evicts if RAM is full.
3. The predictor learns the transition just seen.
4. Every 1,000 requests the decider compares faults with the shadow directory and moves its threshold.
5. The predictor proposes candidates (readahead window, then the Markov guess).
6. Candidates already in RAM are skipped; the decider accepts or rejects each.
7. Accepted candidates are prefetched through the normal install path.
8. At the end, counters are returned and saved; the time model replays the event log.

14.3 Concepts to remember

- Page fault; compulsory fault; working set; eviction policy; OPT
- Prefetching risk: every prefetch into full RAM evicts a page
- Markov (pages), delta-Markov (gaps), stride, readahead (window doubling)
- Confidence floor 0.5 is structural
- Fault delta; oracle vs ghost list vs shadow directory; SHARDS sampling
- Warm-up-only tuning
- Time model: late prefetches, queueing, channels

14.4 Key formulas and metrics

Metric	Formula
vs_LRU	(LRU faults − faults) / LRU faults
useful	coverage × accuracy
confidence	count(best) / total counts
delta rate	(faults without prefetching − faults with) / epoch
break-even c*	(LRU faults − faults) / prefetches
speed-up	time LRU / time system
readahead confidence	run / (run + 1)
page number	address >> 12

14.5 Component → responsibility

Component	Responsibility
Trace	ordered page numbers + 25% warm-up split
MemoryEngine	RAM, faults, the single install path
Eviction policy	which page leaves
Predictor	which pages come next
Decider	whether to prefetch; adapts the threshold
Shadow directory	exact "without prefetching" fault count
Time model	faults and prefetches → estimated time

14.6 Most important files and functions

File	Functions / classes
src/paging/engine.py	MemoryEngine._install / access / prefetch; LRU; OPTFast.choose_victim
src/paging/predictors.py	MarkovPredictor.update / predict; DeltaMarkovPredictor; ReadaheadPredictor.update; StreamMarkovPredictor
src/paging/deciders.py	CostAwareFaultDelta.decide / _adapt; CostAwareGhost; CostAwareDirectory
src/paging/simulate.py	run_prefetch_sim; run_prefetch_sim_ghost; run_prefetch_sim_dir; sampled
src/paging/timing.py · final.py	replay; run_final
scripts/11, 14, 17	robustness, time model, component comparison

14.7 Novelty in two sentences

We show which feedback signal a cost-aware prefetcher should use (the measured fault delta, not confidence or precision) and provide an exact, cheap, samplable way to measure it (a page-ID-only shadow directory). Combined with readahead and a delta-Markov predictor it removes the most faults of every system tested, and our time model shows when that does and does not make programs faster.

14.8 Two-minute explanation

> "Programs keep their working data in RAM and the rest on disk. When they need something that is not in RAM they wait: a page fault. Our project is a simulator that replays real programs' memory requests and tests whether we can avoid those waits by predicting the next pages and loading them early. The catch is that RAM is full, so every early load pushes something else out, and a wrong guess can cause a fault. So we built two parts: predictors (a pattern-learning Markov model, a version that learns gaps, and the readahead Linux uses) and a decision engine that measures every thousand requests whether we are causing fewer faults than without prefetching and becomes bolder or more careful. We found that the usual signals, confidence and precision, mislead, and we measure the real difference cheaply with a directory that stores only page names. On 22 real programs the final system removes about 55% of the faults of LRU, more than Linux readahead. But when we model time, it is about 1.4 times faster on a fast SSD, the same as readahead, and slower on a hard disk, because fewer faults also means more disk reads and late prefetches. That honest result is part of our contribution."

APPENDIX

Glossary

Term	Meaning
ARC	Adaptive Replacement Cache: balances recently and frequently used pages using ghost lists
Break-even cost	Largest prefetch cost (fraction of a fault) at which a system still beats LRU
Clock	Second-chance approximation of LRU
Compulsory fault	Fault on the first ever request for a page
Confidence	Share of the best successor's count in a Markov context
Dead band	Range of delta rates (±0.002) treated as "no change"
Delta-Markov	Markov table over gaps between pages
Epoch	Period between feedback updates (1,000 requests in the final system)
Eviction	Removing a page from RAM
Fault delta	Faults without prefetching − faults with prefetching
FIFO	First-in first-out eviction
Frame	A slot in RAM holding one page
Ghost list	List of IDs of pages a prefetch evicted, to detect harm
Headroom	How much better OPT is than LRU
LLC	Last-level cache: the processor cache the traces were recorded at
LRU / LRU-2	Least recently used; second-last use
OPT (Belady)	Optimal eviction using knowledge of the future
Oracle	Feedback from a full second copy of memory (not deployable)
Page	4 KB block of memory
Precision	Prefetches used / prefetches issued
Prefetch	Loading a page before it is requested
Readahead	Fetching the next pages of a sequential run
SHARDS	Hash-based sampling of pages to shrink tracking structures
Shadow directory	No-prefetch copy of the policy holding page IDs only
Speed-up	time(LRU) / time(system)
Threshold	Minimum confidence accepted when RAM is full
Trace	Recorded list of requested pages
vs_LRU	Fraction of LRU's faults removed
Warm-up portion	First 25% of a trace, the only data used for choosing settings
Working set	Distinct pages a program uses

References and sources

Source	What it supports
src/paging/*.py, tests/test_core.py	All logic and code in Chapters 5-7 (snippets read directly)
results/*.csv (scripts 05-17)	All results in Chapter 10
docs/RESULTS.md (sections 1-17)	Every number with its provenance, including Colab-era results
docs/DECISIONS_AND_LESSONS.md, docs/ROADMAP.md	Design decisions, corrected claims, phase plan
docs/REVIEW_PREP.md, docs/paper_draft.md	Framing, related work, likely reviewer questions
Joseph and Grunwald, "Prefetching using Markov predictors", ISCA 1997	Markov prefetching
Megiddo and Modha, "ARC", FAST 2003; O'Neil et al., "LRU-K", SIGMOD 1993	ARC and LRU-2 baselines
Wu et al., "PACMan", MICRO 2011; Linux mm/workingset.c	Prefetch-aware insertion; shadow entries
Waldspurger et al., "SHARDS", FAST 2015	Hash sampling of the shadow directory
ML-DPC LoadTraces via github.com/Quangmire/ChampSim (download_links)	Real traces